

廈門大學



信息学院软件工程系

《计算机网络》实验报告

题 目	实验六 CISCO IOS 路由器基本配置
班 级	软件工程 2024 级 1 班
姓 名	李富悦
学 号	22920242203267
实验时间	2026 年 5 月 4 日

2026 年 5 月 4 日

1 实验目的

1. 理解网络层路由转发的基本过程，掌握路由器接口地址、运行配置和路由表的查看方法。
2. 使用 Cisco IOS 风格命令完成接口配置、静态路由配置和 RIP v2 动态路由配置，并通过端到端连通性验证配置结果。
3. 通过 VLAN 端口划分和 802.1Q Trunk 子接口实验，理解虚拟局域网隔离与跨 VLAN 三层转发过程。

2 实验环境

本机未安装 Router eSIM、CCNA Network Visualizer 或 Cisco Packet Tracer，因此采用 Docker + FRRouting 的 vtysh 环境复现 Cisco IOS 风格配置命令，并在真实容器网络中验证路由与 VLAN 行为。

操作系统：macOS 26.4.1，Apple Silicon arm64。

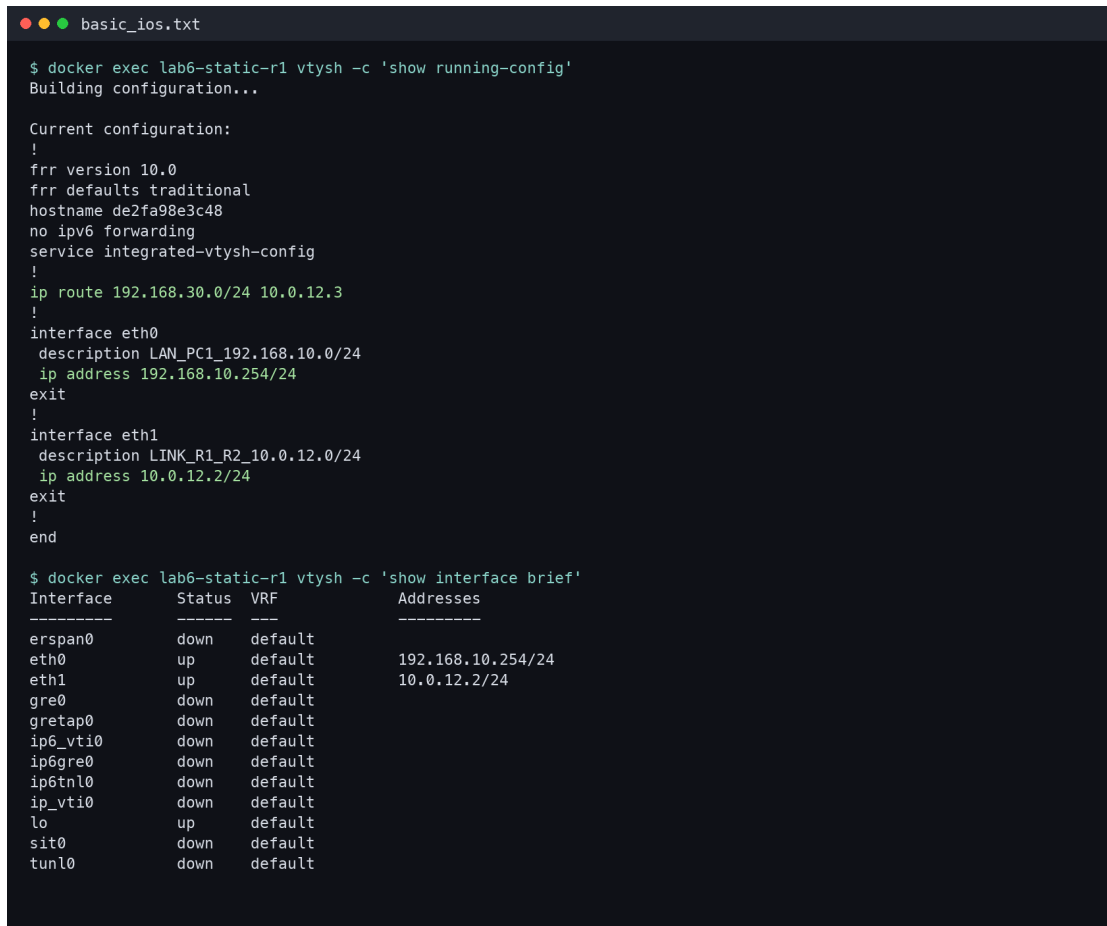
容器环境：Docker Desktop 29.2.1；实验镜像为程序自动构建的 lab6-frr-native:latest，基础镜像 Alpine 3.20，FRRouting 10.0。

编程语言与工具：C++17，clang++，Makefile。核心程序位于 ex6/main.cpp，运行后自动生成 artifacts 目录中的配置、日志和截图取证文本。

3 实验结果

本次实验由 C++ 程序真实创建三类拓扑。静态路由拓扑包含 PC1 - R1 - R2 - R3 - PC2，网段分别为 192.168.10.0/24、10.0.12.0/24、10.0.23.0/24 和

192.168.30.0/24 ; RIP 拓扑使用 192.168.110.0/24、10.1.12.0/24、10.1.23.0/24 和 192.168.130.0/24 ; VLAN 拓扑使用 Linux bridge 的 vlan_filtering 模式模拟交换机接入端口和 Trunk 端口。



```
basic_ios.txt

$ docker exec lab6-static-r1 vtysh -c 'show running-config'
Building configuration...

Current configuration:
!
frr version 10.0
frr defaults traditional
hostname de2fa98e3c48
no ipv6 forwarding
service integrated-vtysh-config
!
ip route 192.168.30.0/24 10.0.12.3
!
interface eth0
 description LAN_PC1_192.168.10.0/24
 ip address 192.168.10.254/24
exit
!
interface eth1
 description LINK_R1_R2_10.0.12.0/24
 ip address 10.0.12.2/24
exit
!
end

$ docker exec lab6-static-r1 vtysh -c 'show interface brief'
Interface      Status VRF      Addresses
-----
erspan0        down   default
eth0            up     default  192.168.10.254/24
eth1            up     default  10.0.12.2/24
gre0           down   default
gretap0        down   default
ip6_vti0       down   default
ip6gre0        down   default
ip6tnl0        down   default
ip_vti0        down   default
lo              up     default
sit0           down   default
tunl0          down   default
```

图 1 R1 的运行配置与接口状态，eth0 和 eth1 已配置 IP 地址并处于 up 状态。

图 1 显示 R1 已配置 192.168.10.254/24、10.0.12.2/24 和到 192.168.30.0/24 的静态路由，show interface brief 可见两个实验接口均处于 up 状态。

```
static_routes.txt

$ docker exec lab6-static-r1 vtysh -c 'show ip route'
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 192.168.10.1, eth0, 00:00:15
C>* 10.0.12.0/24 is directly connected, eth1, 00:00:15
L>* 10.0.12.2/32 is directly connected, eth1, 00:00:15
C>* 192.168.10.0/24 is directly connected, eth0, 00:00:15
L>* 192.168.10.254/32 is directly connected, eth0, 00:00:15
S>* 192.168.30.0/24 [1/0] via 10.0.12.3, eth1, weight 1, 00:00:05

$ docker exec lab6-static-r2 vtysh -c 'show ip route'
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 10.0.12.1, eth0, 00:00:12
C>* 10.0.12.0/24 is directly connected, eth0, 00:00:12
L>* 10.0.12.3/32 is directly connected, eth0, 00:00:12
C>* 10.0.23.0/24 is directly connected, eth1, 00:00:12
L>* 10.0.23.2/32 is directly connected, eth1, 00:00:12
S>* 192.168.10.0/24 [1/0] via 10.0.12.2, eth0, weight 1, 00:00:06
S>* 192.168.30.0/24 [1/0] via 10.0.23.3, eth1, weight 1, 00:00:06

$ docker exec lab6-static-pc1 ping -c 4 -W 2 192.168.30.10
PING 192.168.30.10 (192.168.30.10) 56(84) bytes of data.
64 bytes from 192.168.30.10: icmp_seq=1 ttl=61 time=0.172 ms
64 bytes from 192.168.30.10: icmp_seq=2 ttl=61 time=0.268 ms
64 bytes from 192.168.30.10: icmp_seq=3 ttl=61 time=0.143 ms
64 bytes from 192.168.30.10: icmp_seq=4 ttl=61 time=0.174 ms

--- 192.168.30.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3080ms
rtt min/avg/max/mdev = 0.143/0.189/0.268/0.047 ms
```

图 2 静态路由实验中 R1/R2 的路由表和 PC1 到 PC2 的连通性测试。

图 2 中 R1 的路由表出现 S>* 192.168.30.0/24 via 10.0.12.3 , R2 同时具有去往 192.168.10.0/24 和 192.168.30.0/24 的静态路由。PC1 ping 192.168.30.10 发送 4 个报文全部收到回复, 丢包率为 0%。

```

rip_routes.txt

$ docker exec lab6-rip-r1 vtysh -c 'show ip rip' -c 'show ip route'
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
  (n) - normal, (s) - static, (d) - default, (r) - redistribute,
  (i) - interface

   Network          Next Hop      Metric From      Tag Time
C(i) 10.1.12.0/24    0.0.0.0       1 self           0
R(n) 10.1.23.0/24    10.1.12.3     2 10.1.12.3       0 02:56
C(i) 192.168.110.0/24 0.0.0.0       1 self           0
R(n) 192.168.130.0/24 10.1.12.3     3 10.1.12.3       0 02:56
Codes: K - kernel route, C - connected, L - local, S - static,
R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
f - OpenFabric, t - Table-Direct,
> - selected route, * - FIB route, q - queued, r - rejected, b - backup
t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 192.168.110.1, eth0, 00:00:19
C>* 10.1.12.0/24 is directly connected, eth1, 00:00:19
L>* 10.1.12.2/32 is directly connected, eth1, 00:00:19
R>* 10.1.23.0/24 [120/2] via 10.1.12.3, eth1, weight 1, 00:00:04
C>* 192.168.110.0/24 is directly connected, eth0, 00:00:19
L>* 192.168.110.254/32 is directly connected, eth0, 00:00:19
R>* 192.168.130.0/24 [120/3] via 10.1.12.3, eth1, weight 1, 00:00:04

$ docker exec lab6-rip-r3 vtysh -c 'show ip route'
Codes: K - kernel route, C - connected, L - local, S - static,
R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
f - OpenFabric, t - Table-Direct,
> - selected route, * - FIB route, q - queued, r - rejected, b - backup
t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 192.168.130.1, eth1, 00:00:13
R>* 10.1.12.0/24 [120/2] via 10.1.23.2, eth0, weight 1, 00:00:05
C>* 10.1.23.0/24 is directly connected, eth0, 00:00:13
L>* 10.1.23.3/32 is directly connected, eth0, 00:00:13
R>* 192.168.110.0/24 [120/3] via 10.1.23.2, eth0, weight 1, 00:00:05
C>* 192.168.130.0/24 is directly connected, eth1, 00:00:13
L>* 192.168.130.254/32 is directly connected, eth1, 00:00:13

$ docker exec lab6-rip-pc1 ping -c 4 -W 2 192.168.130.10
PING 192.168.130.10 (192.168.130.10) 56(84) bytes of data.
64 bytes from 192.168.130.10: icmp_seq=1 ttl=61 time=0.145 ms
64 bytes from 192.168.130.10: icmp_seq=2 ttl=61 time=0.288 ms
64 bytes from 192.168.130.10: icmp_seq=3 ttl=61 time=0.112 ms
64 bytes from 192.168.130.10: icmp_seq=4 ttl=61 time=0.133 ms

--- 192.168.130.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3082ms
rtt min/avg/max/mdev = 0.112/0.169/0.288/0.069 ms

```

图 3 RIP v2 动态路由实验中的路由学习结果和端到端连通性测试。

图 3 中 R1 通过 RIP 学到 10.1.23.0/24 和 192.168.130.0/24，R3 通过 RIP 学到 10.1.12.0/24 和 192.168.110.0/24，路由项标记为 R>*，管理距离为 120。PC1 ping 192.168.130.10 的丢包率为 0%。

```

vlan.txt

$ docker exec lab6-vlan-lab bridge vlan show
port          vlan-id
br0           1 PVID Egress Untagged
swp1          10 PVID Egress Untagged
swp2          20 PVID Egress Untagged
swp3          10
              20

$ docker exec lab6-vlan-lab ip netns exec router ip -d link show rtr-eth0.10
11: rtr-eth0.10@rtr-eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group
default qlen 1000
    link/ether fe:20:ea:ba:85:cb brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0 minmtu 0 maxmtu 65535
    vlan protocol 802.1Q id 10 <REORDER_HDR> addrgenmode eui64 numtxqueues 1 numrxqueues 1 gso_max_size 65536
    gso_max_segs 65535 tso_max_size 524280 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536
    gro_ipv4_max_size 65536

$ docker exec lab6-vlan-lab ip netns exec router ip route
172.16.10.0/24 dev rtr-eth0.10 proto kernel scope link src 172.16.10.1
172.16.20.0/24 dev rtr-eth0.20 proto kernel scope link src 172.16.20.1

$ docker exec lab6-vlan-lab ip netns exec vlan10-host ping -c 4 -W 2 172.16.20.10
PING 172.16.20.10 (172.16.20.10) 56(84) bytes of data:
64 bytes from 172.16.20.10: icmp_seq=1 ttl=63 time=0.031 ms
64 bytes from 172.16.20.10: icmp_seq=2 ttl=63 time=0.097 ms
64 bytes from 172.16.20.10: icmp_seq=3 ttl=63 time=0.037 ms
64 bytes from 172.16.20.10: icmp_seq=4 ttl=63 time=0.116 ms

--- 172.16.20.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3050ms
rtt min/avg/max/mdev = 0.031/0.070/0.116/0.036 ms

```

图 4 VLAN 10/20 接入端口、Trunk 端口和跨 VLAN 三层转发验证。

图 4 中 swp1 为 VLAN 10 access 端口，swp2 为 VLAN 20 access 端口，swp3 同时承载 VLAN 10 和 VLAN 20。router 命名空间内的 rtr-eth0.10 使用 802.1Q id 10，路由表包含 172.16.10.0/24 和 172.16.20.0/24，VLAN 10 主机 ping VLAN 20 主机的丢包率为 0%。

4 实验代码

本次实验代码位于代码仓库：<https://github.com/lifuyue/xmu-cn-lab/tree/main/ex6>。

程序入口为 main.cpp，编译目标为 ios_router_lab。运行 make && ./ios_router_lab 后，程序会自动构建实验镜像、清理旧容器、创建 Docker 网络下发 IOS 风格配置命令、执行 ping 验证，并保存 show running-config、show ip route、show ip rip、bridge vlan show 等真实输出。

代码片段 1：C++ 程序通过 vtysh 发送 IOS 风格配置命令。

```
void docker_exec_vtysh(const std::string &container, const std::vector<std::string>
&commands) {
    const std::string command = "docker exec " + container + " vtysh" +
join_commands(commands);
    require_success(command);
}
```

代码片段 2：启动 FRRouting 守护进程时加入重试，避免实验受容器启动瞬时状态影响。

```
start_one    /var/run/frr/mgmt.d.vty    /var/run/frr/mgmt.d.pid    /usr/lib/frr/mgmt.d
/tmp/mgmt.d.log &&
start_one /var/run/frr/zebra.vty /var/run/frr/zebra.pid /usr/lib/frr/zebra /tmp/zebra.log
&&
start_one    /var/run/frr/static.d.vty    /var/run/frr/static.d.pid    /usr/lib/frr/static.d
/tmp/static.d.log
```

代码片段 3：静态路由实验的 R1 核心配置。

```
interface eth0
    ip address 192.168.10.254/24
interface eth1
    ip address 10.0.12.2/24
ip route 192.168.30.0/24 10.0.12.3
```

5 课后思考题

1. 怎样对路由器设置密码保护？

可以在特权模式下配置 enable secret 保护特权模式，在 line console 0 下配置 password 和 login 保护 Console 登录，在 line vty 0 4 下配置 password/login 或 login local 保护远程登录，并使用 service password-encryption 对明文口令做基本加密。

2. 利用 show 命令如何判定网络故障？

先用 `show ip interface brief` 查看接口是否 up/up，再用 `show interfaces` 查看错误包、丢包和链路状态；用 `show ip route` 查看是否存在到目的网段的路由；用 `show running-config` 核对接口地址、shutdown 状态和路由协议配置；动态路由场景下还可用 `show ip rip` 或对应协议的 show 命令检查是否学到路由。

3. 路由器有几种配置模式？如何在各个配置模式之间切换？

常见模式包括用户模式 Router>、特权模式 Router#、全局配置模式 Router(config)#、接口配置模式 Router(config-if)#、线路配置模式 Router(config-line)# 和路由协议配置模式 Router(config-router)#。从用户模式输入 enable 进入特权模式，输入 configure terminal 进入全局配置模式，输入 interface、line 或 router rip 进入对应子模式，输入 exit 退回上一级，输入 end 或 Ctrl-Z 返回特权模式。

4. 如何判定路由器的串口是广域网的 DTE 端还是 DCE 端？

在真实 Cisco IOS 中可使用 `show controllers serial 接口号` 查看串口线缆类型输出中会标明 DTE 或 DCE。DCE 端需要提供时钟，一般要配置 clock rate；DTE 端不配置时钟速率。

6 实验总结

本次实验完成了 IOS 风格基础配置、静态路由、RIP 动态路由和 VLAN 划分验证。通过真实容器网络中的路由表和 ping 输出，可以直观看到静态路由依赖人工指定下一跳，而 RIP 能够通过邻接路由器交换可达信息自动生成路由项。VLAN 部分进一步验证了二层隔离和三层子接口转发的关系。实现过程中

也处理了 FRRouting 守护进程启动顺序和本机 arm64 环境兼容性问题，使实验过程能够由 C++ 程序稳定复现。